HAIL MARY .RO

Handoff spec for Claude Code — Next.js build

1. Stack decisions

- **Framework:** Next.js 14+ (App Router), TypeScript
- **Styling:** CSS Modules + Sass (`.module.scss` per component, no Tailwind, no global class soup)
- **Content:** MDX files in-repo under `content/articles/<locale>/` (frontmatter + body). No headless CMS for v1 — keeps the portfolio build self-contained and dependency-free; CMS swap is a documented future step, not a blocker.
- **i18n:** `next-intl` with locale-prefixed routing (`/ro`, `/en`), `ro` as default. Scope: bilingual UI and evergreen reference content; news stays Romanian-only. Adopted at scaffold time — retrofitting `/[locale]/` later touches every route, link and `generateStaticParams`.
- **State:** no global store. Server state via RSC + ISR, shared UI state in Context (`TeamColorProvider`, `ExplainerProvider`), local state in `useState`, live remote data via TanStack Query. See AGENT.md.
- **Live data:** a typed fetch layer around a sports-data API (schedules/scores), cached via Next.js ISR (`revalidate`), refreshed by a cron-triggered route
- **Hosting:** Hostinger Business plan (Node.js app deployed from GitHub, builds on push) at hailmary.ro — full Next.js runtime, so SSR, ISR, API routes and middleware are all available. A parallel Vercel deploy off the same repo provides the portfolio link and per-PR previews. No host-specific code paths: the app runs unmodified on both.

2. Page list (App Router routes)

Route	Purpose
/	Homepage — hero story, news grid, beginner's guide sidebar, schedule sidebar
/stiri	News index — paginated list of all articles
/stiri/[slug]	Article detail page (MDX render)
/etichete/[tag]	Tag landing page — all articles carrying one topic tag. generateStaticParams from the controlled list, so only real tags resolve
/echipe	Teams index — 32 NFL teams, logos, division groupings
/echipe/[team]	Team detail — roster highlights, schedule, recent news filtered by team tag
/regulament	Rules explainer (evergreen reference)
/istorie	History of American football (evergreen reference)
/glosar	Glossary of terms, alphabetical, searchable client-side
/program	Schedule — full week/season schedule, live score states

3. Component breakdown

One component + one .module.scss per item below. Shared primitives live in components/ui/; page-specific composites live in components/<page>/.

- **Layout**: `SiteHeader` (logo, nav, team-color picker — no account CTA), `SiteFooter`, `TeamColorProvider` (context wrapping the 6-color accent system — extendable to all 32 teams), `OriginStrip` (dismissible 76px band under the header telling the Hail Mary origin story)
- **Homepage**: `HeroStory`, `NewsGrid`, `NewsCard`, `BeginnersGuideSidebar`, `ScheduleSidebar`
- **Articles**: `ArticleCard` (grid item, reused on `/stiri`), `ArticleBody` (MDX renderer wrapper w/ typography), `ArticleMeta` (byline/date/category chip), `RelatedArticles`
- **Teams**: `TeamCard`, `TeamBadge` (logo + accent color swatch), `DivisionGroup`
- **Reference**: `GlossaryTerm`, `GlossaryList` (with client-side filter), `RuleSection`, `TimelineEntry` (for `/istorie`)
- **Explainer panel**: `ExplainerProvider` (context: open state + active term), `ExplainerPanel` (the slide-in panel itself), `ExplainerContent` (renders one term's extended explanation), `TermTooltip` (desktop hover tier, shows `short`), `TermLink` (the inline trigger inside article body copy)
- **Schedule/Data**: `ScheduleTable`, `GameRow`, `LiveScoreBadge`
- **Shared UI**: `Button`, `Tag`, `SectionHeading`

4. Data shapes (TypeScript)

```
// content/articles/<locale>/*.mdx frontmatter
type ArticleFrontmatter = {
  slug: string;
  title: string;
  excerpt: string;
  coverImage: string;
  publishedAt: string;       // ISO date
  category: 'stiri' | 'regulament' | 'istorie' | 'glosar';  // content
  teams?: string[];          // who it's about — team slugs, e.g. ['kc
  tags?: Tag[];              // what it's about — controlled vocabular
  featured?: boolean;
};

// lib/tags.ts — the only place tags are defined. A typo fails the bui
const TAGS = ['transferuri','accidentari','analiza','antrenori',
              'draft','playoffs','super-bowl'] as const;
type Tag = typeof TAGS[number];
```

```
type Team = {
  slug: string;              // 'kc'
  name: string;               // 'Kansas City Chiefs'
  conference: 'AFC' | 'NFC';
  division: string;           // 'West'
  accent1: string;            // hex
  accent2: string;            // hex
  logoUrl: string;
};

type Game = {
  id: string;
  week: number;
  season: number;
  kickoff: string;            // ISO datetime
  homeTeam: string;           // team slug
  awayTeam: string;
  homeScore?: number;
  awayScore?: number;
  status: 'scheduled' | 'live' | 'final';
};

type GlossaryEntry = {
  slug: string;              // 'play-action', used by TermLink
  term: string;              // 'Play action'
  short: string;             // one sentence — glossary list + tooltip
  extended: string;          // MDX string — the panel body
  relatedTerms?: string[];   // other entry slugs
  seeAlso?: string;          // route, e.g. '/regulament#pase'
};
```

5. SCSS folder structure

```
styles/
  _variables.scss     // color tokens (base + 6 team accent pairs), sp
  _mixins.scss        // breakpoints, truncation, focus-ring
  globals.scss        // resets, font-face, body defaults (imported on

components/
  ui/
    Button/Button.tsx + Button.module.scss
    Tag/Tag.tsx + Tag.module.scss
  layout/
    SiteHeader/SiteHeader.tsx + SiteHeader.module.scss
    SiteFooter/SiteFooter.tsx + SiteFooter.module.scss
  home/
    HeroStory/…
    NewsGrid/…
  articles/
    ArticleCard/…
    ArticleBody/…
  teams/
    TeamCard/…
  reference/
    GlossaryList/…
  explainer/
    ExplainerPanel/…
    TermLink/…
  schedule/
    ScheduleTable/…
```

Each component folder co-locates its `.tsx` and `.module.scss`. Team accent colors are CSS custom properties set on a root wrapper by `TeamColorProvider`, referenced in SCSS via `var(--accent-1)` — this is what lets "Echipa mea" re-skin the whole layout without per-component logic.

6. Content workflow — adding an article

1. Add a new `.mdx` file to `content/articles/ro/`, named `<slug>.mdx`. News is Romanian-only — no `en/` twin; evergreen reference content does get one, at the same slug
2. Fill in frontmatter (title, excerpt, cover image, category, team tags, date)
3. Write body in MDX (supports embedding `<ScheduleTable>`, callouts, images inline)
4. Next.js's static generation (`generateStaticParams`) picks it up at build time; ISR revalidates on a schedule so new articles surface without a full redeploy
5. **Future CMS step** (documented, not built for v1): swap the MDX file read for a headless CMS query (e.g. Sanity) behind the same `getArticleBySlug`/`getAllArticles` interface — no component changes needed

7. Live data — schedule & scores

- A cron-triggered Next.js route (`/api/cron/sync-scores`) calls the sports-data API on a schedule (e.g. every 60s during live windows, hourly otherwise)
- Hostinger has no managed cron, so the trigger is an **hPanel cron job** hitting the route over HTTPS. The route is therefore a public URL: it authenticates on a `CRON_SECRET` header, 401s otherwise, and must be idempotent and safe under overlapping invocations
- Writes normalized `Game` records to a small store (start with a JSON/KV cache; swap to a database if traffic justifies it)
- `ScheduleTable`/`LiveScoreBadge` read from that store via ISR, not directly from the third-party API on every request

8. Origin strip

A dismissible band directly under the header, matching its height (~76px), that answers the question a first-time visitor actually has: why is the site called Hail Mary? Small field diagram on the left with a ball arcing once, the story revealed in four short phrases over ~4.5s, a link into the glossary entry, and a close button.

- Copy is four phrases, cumulative (each stays once revealed) — the reader ends on the full paragraph, so nothing depends on catching the animation
- The ball's arc is timed to the throw phrase (~2.65s), not to load. If the animation and the sentence drift apart, the sentence wins
- Respect `prefers-reduced-motion`: render the full paragraph immediately, no reveal, no ball
- Dismissal persists in `localStorage` (seen once, never again). Read it before first paint — an inline script setting a class on `<html>`, or render server-side hidden and reveal — so a returning reader never sees it flash
- The band must not push layout on dismiss in a way that shifts the hero mid-read; it is removed from flow entirely
- Text may wrap to two lines but not three at 1280px. If a translation overflows, shorten the copy rather than growing the band

9. Explainer side panel

The feature that makes one article serve both audiences: tagged jargon in body copy opens a slide-in panel with an extended explanation, so a beginner gets depth on demand while a fan reading straight through is never interrupted. This is the site's differentiator — build it before any widget work.

Authoring syntax

In article MDX, wrap the term. The child text is what renders inline, so it can be inflected to fit the sentence while `term` stays the canonical entry slug:

```
Chiefs au câștigat cu un <TermLink term="play-action">play action</Ter
în ultimul sfert.
```

`TermLink` is registered in the MDX component map, so no per-article import. An unknown `term` fails the build rather than rendering a dead trigger — a typo in a slug must never ship as a silently inert word.

Two tiers: hover shows, click opens

The two content lengths in a glossary entry map to two levels of interaction, so a reader can get a one-line answer without losing their place, and full depth only if they want it.

	Desktop	Mobile
Hover	Tooltip with `short`	—
Click / tap	Side panel with `extended`	Bottom sheet with `extended`

- **No "show more" button in the tooltip.** A button inside a hover tooltip forces the cursor to travel from the word into a floating box across dead space, which needs dismiss delays and a safe-triangle hit area to survive a trackpad — real effort for a worse interaction than clicking the word. The tooltip instead carries a quiet hint at its foot (*„Click pentru explicația completă"*), which does a button's job at no cost.
- Click is the primary gesture and the **same on both platforms**; hover is a desktop-only enhancement layered on top, never a required step to reach the panel.
- Tooltip appears after a ~300ms hover delay and dismisses immediately on mouse-out — it must never intercept a click aimed at the word, and must not appear at all for keyboard focus (focus goes straight to the panel on Enter/Space).
- Mobile has no hover tier: a tap goes straight to the sheet. Deliberately *not* tap-for-tooltip-then-tap-again — that makes the common case cost two taps to serve the rare one.
- The tooltip is presentational only: no focus trap, no links, no `relatedTerms` chips. Everything interactive lives in the panel.

Panel behaviour

- **Desktop** (≥1024px): panel slides in from the right, ~380px, article column shifts left rather than being covered — reading position is never lost
- **Mobile**: bottom sheet, ~70vh, with a backdrop
- Opening a second term while the panel is open swaps content in place; no close-and-reopen
- Closes on Escape, backdrop click, and an explicit close button. Focus moves into the panel on open and returns to the triggering term on close
- The active term is marked in the body copy while its panel is open, so the reader can see what they clicked
- `relatedTerms` render as chips at the panel foot and swap panel content in place; `seeAlso` is a real navigation link out to the rules or history page
- Panel state lives in `ExplainerProvider` (React Context — `{activeTerm, open, close}`). Not a global store; see AGENT.md on state management
- Deep-linkable: `?termen=play-action` opens the panel on load, so an explanation can be shared or linked from elsewhere

Content source

- One source of truth: `content/glossary/<locale>/<slug>.mdx`, with `short` in frontmatter and the file body as `extended`
- The same entries render three ways — the `/glosar` page (full list), the panel (extended), and a hover tooltip on desktop (`short`). Never a second copy of a definition
- Glossary is part of the evergreen third, so it **is** translated to English (unlike news) — see the localization scope in AGENT.md
- Entries are static content, bundled at build time. The panel must never fetch on open — it appears instantly or the feature is pointless

Degradation

Without JS, `TermLink` renders as an anchor to `/glosar#<slug>`. The explanation is always reachable; the panel is the enhancement.